ICT2232 SOFTWARE ENGINEERING MINI PROJECT

HomeLink.

Home services, thoughtfully connected

Group 3 | Engineering report & appendices

Laravel • Requirements • Design • Testing • Maintenance

Prepared 16 September 2026

Member names, student IDs and contributions: supplied by the group

Local demo verified. Lecturer approval and external acceptance pending.

Reading this report

This report completes the technical plan and documents the delivered HomeLink implementation. The original assignment and incomplete proposal are preserved under references/. Editable requirements, diagrams, tests, deployment instructions and contribution records are included in docs/.

Delivery at a glance

Application	Verification	Academic evidence
Three roles; service discovery; booking lifecycle; reviews; cancellation	88 passing tests; 241 assertions; browser acceptance checks; local deployment	Requirements, 14 stories, design models, traceability, risk register, maintenance record and demo guide

Evidence boundaries

The source and local Git history identify this as AI-assisted implementation. The group must supply its real names, student IDs, individual work and stakeholder evidence. CR-01 is implemented as a candidate maintenance change; lecturer approval is pending. Docker and public hosting have not been executed.

Contents

1. Project report and SDLC evidence
2. Requirements and use cases
3. User stories and Agile plan
4. Rendered design diagrams
5. Security and risk assessment
6. Maintenance change CR-01
7. Test strategy and results

HomeLink

Group 3 - ICT2232 Software Engineering Mini Project

University of Ruhuna · Department of Information and Communication Technology Report date: 16 September 2026 Prepared for Group 3. Five member names and student numbers: to be supplied by the group.

1. Introduction and problem domain

HomeLink connects households with home-service providers such as electricians, plumbers and cleaners. The provided proposal identifies fragmented discovery through personal contacts and separate communication channels as the central problem. Customers need a clear way to compare available services and coordinate a visit. Providers need a place to explain their work and organize incoming requests.

This implementation continues the proposal’s HomeLink identity, Laravel choice, three user roles, Agile process and 17 functional requirement identifiers. It completes the application and supplies the missing engineering artifacts. The assignment is a software-engineering exercise covering the full SDLC, not only a visual prototype.

2. Problem statement

A customer may struggle to identify an appropriate provider, learn what a service includes, agree a time and track a request. A provider may struggle to present services consistently or see overlapping commitments. Information scattered across different channels makes errors and misunderstandings more likely.

HomeLink centralizes service information and booking state. Its main workflow is discover -> request -> accept -> start -> complete -> review. A separate maintenance increment adds eligible customer cancellation.

3. Stakeholders and objectives

Stakeholder	Need	Delivered response
Customer	Find relevant expertise and follow a request	Filters, details, booking form, history and timeline
Provider	Present work and organize incoming requests	Profile, owned listings, status workflow
Administrator	Operate the platform and manage misuse	Account search, suspension and reactivation
Group 3	Understand and explain an SDLC implementation	Source, tests, design, traceability and demo guide
Lecturer	Assess decisions, working behavior and contribution	Local deployment package and engineering evidence

Objectives are to support the proposal's core requirements; keep records persistent; prevent access to other users' private bookings; reject conflicting provider slots; maintain a useful status history; and provide reproducible setup and tests. Interview-derived validation is still for the group to perform. These stakeholder needs come from the supplied proposal and design analysis, not from invented interviews.

4. Scope, assumptions and constraints

Included: customer/provider accounts, profiles, service categories, service discovery, provider listing management, booking lifecycle, administration, validation, ownership controls, reviews and cancellation.

Excluded: online payment collection, live chat, email/SMS delivery, GPS, native mobile apps, advanced AI recommendations, smart-home integration, background checks, enterprise accounting and multi-region operations. The proposal left some of these undecided. Reviews were selected as a feasible enhancement; payments remain direct between customer and provider.

Assumptions: one provider performs one two-hour appointment at a time; service cities are provider-entered labels; all times use Asia/Colombo; customers and providers communicate using the booking contact details; the listed price is a starting quote, not an online payment charge. Providers keep descriptions and availability accurate. Slots begin at 08:00, 10:00, 12:00, 14:00 and 16:00 and may be requested up to 90 days ahead.

Constraints include academic time, five team members, a Windows demonstration computer, and limited external service setup. SQLite makes the local demo self-contained; MySQL configuration remains supported by Laravel but has not been integration-tested for this delivery. Docker is specified for reproducible team environments but is not installed on the inspected computer.

5. Process model and project planning

The proposal selects an iterative Agile model. This is appropriate because identity, service management, discovery, booking and administration can each produce a usable increment. Early tests reduce integration surprises. A small modular monolith keeps those modules in one deployable application.

The proposed five-sprint plan is in AGILE-PLAN.md. It is a planning artifact, not a claim that the group held those sprints. The actual local commit history records AI-assisted implementation, a tested baseline and subsequent changes. Team members should develop and review their own follow-up work under their own identities.

6. Requirements engineering

REQUIREMENTS.md preserves FR-01 to FR-17 and NFR-01 to NFR-07. Additional implemented requirements cover reviews, cancellation and completion tracking. It defines business rules, use-case descriptions, MoSCoW priorities and links to implementation and test evidence.

AGILE-PLAN.md expands the proposal's four fully described stories to fourteen, including the story identifiers already referenced in its unfinished traceability matrix. Acceptance criteria describe observable outcomes. Stakeholders should validate the requirements before treating them as a formally approved baseline.

7. System design

The browser receives server-rendered Blade pages. Alpine handles local interface state and availability requests. GSAP supplies progressive motion; reduced-motion preferences disable decorative animation. Vite builds local CSS, JavaScript and fonts.

Laravel web middleware manages sessions, request-forgery protection and authentication. Role middleware restricts workspaces; policies restrict individual records. Controllers validate HTTP input and coordinate application operations. BookingService owns transactional booking creation, state transitions and cancellation. Eloquent models persist domain records.

DESIGN.md contains architecture, use cases, a class/domain model, an ER diagram, two sequence diagrams, an activity diagram and wireframes. Designs reflect the implemented system, including soft-deleted services and quoted-price snapshots.

8. Technology decisions

Technology	Reason
Laravel 13 / PHP 8.4	Requested framework; built-in validation, sessions, policies, migrations, testing and ORM
Blade	Direct mapping between routes and accessible pages; simple for a student team to explain
Alpine.js	Small interactive controls without a separate frontend API application
Tailwind CSS 4 and component CSS	Responsive foundations plus a consistent bespoke visual system
GSAP	Sequenced entrance effects and progressive card motion with reduced-motion support
Lucide and local font files	Consistent icons and typography without runtime CDN dependencies
SQLite	Persistent, portable, zero-service local demonstration
Vite / npm lockfile	Reproducible frontend asset builds
PHPUnit and Pint	Automated behavior checks and consistent PHP formatting
Docker Compose	Supplied portable deployment definition; execution remains to be validated
Git and GitHub Actions	Local history and a ready CI workflow; no remote repository was created

Version locks are authoritative. More dependencies were not added simply to increase the technology count.

9. Implementation

Three roles have separate capabilities. Public visitors may browse and read service details. A customer can request a slot, inspect their booking timeline and review a completed booking. A provider can maintain their own listings and move their bookings through the allowed stages. An administrator can manage ordinary

accounts but cannot suspend administrators.

Booking creation validates the request, reloads the listing inside a transaction, stores a price/title snapshot, reserves a unique provider/time slot and creates the first history event. A conflict rolls back every part of the request. The unique database constraint is the final guard against competing writes; SQLite does not provide the same row-lock behavior as MySQL.

Removing a listing soft-deletes it. Existing bookings retain the relationship and price snapshot, preserving history. Suspending a user prevents the next authenticated request and hides that provider's services. Reviews require a completed booking, the matching customer, and a unique booking ID.

Fictional sample data is deliberately labeled. The provider profile indicator does not claim professional identity verification. All photographs are illustrative.

10. Testing and quality assurance

The tested baseline passed 79 tests with 213 assertions. The cancellation increment added nine cases, resulting in 88 passing tests and 241 assertions. Unit tests cover status transition rules; HTTP feature tests integrate routing, middleware, validation, policies, services and the real in-memory SQLite database.

Tests include valid and invalid registration, prohibited administrator registration, account suspension, login throttling, profile privilege injection, forged booking prices, duplicate provider slots, invalid dates, unauthorized record access, invalid state transitions, completed-only reviews and cancellation boundaries. TESTING.md documents the plan, test cases, defects and evidence paths.

Browser checks exercise the live application using fictional credentials, including request creation and provider processing. Mobile and desktop visual inspection supplements the automated tests. This is not independent stakeholder acceptance or a production load test.

11. Deployment

The delivered application runs locally at http://127.0.0.1:8000 through a project-local PHP runtime. The Windows launchers start and stop its background development server. Runtime files, secrets and dependencies are excluded from Git.

The Docker recipe provides PHP/Apache, compiled frontend assets and persistent named volumes. It has not been executed on this computer. A public deployment needs a chosen host, TLS, production credentials, backups and a separate validation run. DEPLOYMENT.md gives the complete installation and release procedure and differentiates tested local setup from supplied deployment configuration.

12. Maintenance

CR-01 adds customer cancellation of pending/accepted bookings at least two hours before the appointment. It is perfective maintenance: it improves flexibility after the working baseline. The operation releases the reserved slot, writes a timeline event and preserves the booking.

MAINTENANCE.md includes the change, impact analysis, implementation, estimates and regression evidence. CR-01 is an implemented proposed example. The lecturer has not been represented as approving it; the group must obtain approval or implement the actual requested change.

13. Security

SECURITY.md documents threats, likelihood/impact, controls and residual risks. Implemented controls include password hashing, session regeneration, CSRF protection, role middleware, ownership policies, validation, bound queries, escaped output, rate limiting and database uniqueness. At least five relevant threats and three implemented controls are covered.

The local demonstration is bound to loopback. Development credentials are never appropriate for a public deployment. Debug responses are disabled by default. No compliance certification or complete security audit is claimed.

14. Teamwork and contribution

The five-person roster and individual contributions are deliberately unfilled. CONTRIBUTIONS.md provides an evidence-based log and suggested role rotation. Every member should run the app, inspect the requirements, understand one end-to-end path, add a meaningful improvement or verification, commit it under their own identity and prepare for the viva.

AI-assisted source must be reviewed and understood by the group. The generated commits identify their AI-assisted origin. They are not evidence that any named student wrote or reviewed the code.

15. Conclusion and outstanding evidence

HomeLink implements the assigned domain and the proposal’s core workflows with persistent storage, a polished responsive interface and tested access controls. The delivery includes source, local runtime, deployment configuration and SDLC documentation.

The remaining external evidence is a real stakeholder review, member identities and contributions, lecturer-approved maintenance request, team acceptance sign-off and the final presentation. Public hosting, Docker execution and CI execution are explicitly not claimed as completed.

16. References

1. ICT2232_Software_Engineering_Mini_Project.pdf - supplied assignment, 12 pages. 2. HomeLink Project Proposal.pdf - supplied incomplete proposal, 26 pages. 3. Laravel documentation: https://laravel.com/framework/docs/13.x 4. PHPUnit documentation: https://docs.phpunit.de/en/12.5/ 5. PHP Windows downloads: https://www.php.net/downloads.php?os=windows&version=8.4 6. Composer: https://getcomposer.org/download/ 7. Exact installed dependencies: composer.lock and package-lock.json.

17. Appendices

Requirements, Agile plan, design diagrams, test reports, security, maintenance, user/deployment guides, presentation notes, attribution and contribution templates are provided as separate editable files in this directory.

Requirements specification and traceability

Version 1.1 · Group 3 · 16 September 2026

Source: supplied HomeLink proposal. FR-01-FR-17 and NFR-01-NFR-07 retain their original meaning. FR-18-FR-20 are implementation refinements. Status: implemented baseline for stakeholder review, not an invented stakeholder sign-off.

Functional requirements

ID	The system shall…	Priority	Story / use case	Implementation	Main test evidence
FR-01	register customers and providers with role selection; reject public admin registration	Must	US-01 / UC-01	AuthController::register	AuthControllerTest::test_customer_registration_creates_a_hashed_account; test_provider_can_register; test_invalid_registration_is_rejected
FR-02	authenticate active accounts using valid credentials and support logout	Must	US-01 / UC-02	AuthController::login, logout	test_active_user_can_log_in; test_incorrect_password_does_not_authenticate; test_logout_ends_the_authenticated_session
FR-03	direct signed-in users to their role workspace	Must	US-01 / UC-02	DashboardController	HomeLinkTest::test_role_dashboard_displays_relevant_workspace; test_admin_dashboard_redirects_to_administration
FR-04	allow users to view and update their own profile	Should	US-02, US-09 / UC-10	AuthController::profile; profile view	test_profile_updates_ignore_role_and_active_flag_injection
FR-05	allow providers to create listings under predefined categories	Must	US-02 / UC-03	ProviderController::store	ProviderControllerTest::test_provider_can_create_an_owned_service
FR-06	allow providers to edit, pause or remove their own listings	Should	US-02, US-09 / UC-03	ProviderController; ServiceListingPolicy; soft deletes	test_provider_can_edit_and_pause_their_service; test_other_provider_cannot_modify_or_delete_service; test_removing_service_preserves_booking_history
FR-07	expose service categories for public browsing	Must	US-03 / UC-04	CatalogController::index; Category	CatalogControllerTest::test_public_catalog_shows_active_services_and_categories

ID	The system shall…	Priority	Story / use case	Implementation	Main test evidence
FR-08	filter services by search text, category and city, with allowed sort options	Must	US-03 / UC-04	CatalogController::index	test_search_combines_text_category_and_city; test_price_sort_orders_matching_services
FR-09	display service and provider details	Must	US-08 / UC-05	CatalogController::show; service view	test_public_can_view_service_and_provider_details
FR-10	let customers request a valid future service slot with contact/address details	Must	US-04 / UC-06	BookingController::store; BookingService::create	BookingControllerTest::test_customer_creates_pending_booking_with_a_price_snapshot; test_invalid_booking_does_not_reserve_a_slot; test_conflicting_request_across_services_is_rejected_atomically
FR-11	show customers their booking statuses	Must	US-04, US-11 / UC-08	BookingController::index, show	test_customers_see_only_their_own_booking_history
FR-12	show providers incoming requests for their own services	Must	US-05 / UC-07	BookingController::index; DashboardController	HomeLinkTest::test_role_dashboard_displays_relevant_workspace; role and ownership checks
FR-13	allow the assigned provider to accept or reject pending requests	Must	US-05 / UC-07	BookingService::transition; BookingPolicy	test_provider_acceptance_updates_status_and_timeline; test_rejection_releases_the_provider_slot; test_other_provider_cannot_update_booking
FR-14	let administrators list and search registered accounts	Should	US-06 / UC-09	AdminController::index	AdminControllerTest::test_admin_can_view_accounts_and_platform_stats
FR-15	let administrators suspend/reactivate ordinary accounts	Could	US-12 / UC-09	AdminController::toggle; ActiveAccount	test_admin_can_suspend_a_customer; test_admin_can_reactivate_a_suspended_user; test_suspended_existing_session_is_revoked
FR-16	validate submitted data and show recoverable errors without partial writes	Must	US-07 / all	Controller validation; transactions; flash messages	invalid registration/booking/listing tests and duplicate-slot rollback

ID	The system shall…	Priority	Story / use case	Implementation	Main test evidence
FR-17	enforce roles and record ownership on server requests	Must	US-10 / all protected cases	EnsureRole; BookingPolicy; ServiceListingPolicy	test_ordinary_users_cannot_open_admin_tools; test_other_customer_cannot_view_booking_contact_details; role and ownership checks
FR-18	allow one 1-5 star review by the customer on a completed booking	Should	US-13 / UC-11	BookingController::review; Review unique constraint	test_customer_can_review_a_completed_booking_once; test_unfinished_booking_cannot_be_reviewed
FR-19	cancel an owned pending/accepted booking at least two hours ahead and release its slot	Should / CR-01	US-14 / UC-12	BookingService::cancel; BookingPolicy::cancel	CancellationTest, including boundary and rebooking cases
FR-20	allow accepted -> in progress -> completed and record each transition	Should	US-05, US-11 / UC-07	BookingStatus; BookingService; BookingEvent	HomeLinkTest::test_provider_can_finish_an_accepted_job; BookingStatusTest

FR-15 implements the proposal's disable-or-remove alternative through suspension; permanent user deletion is intentionally not supplied because historical bookings reference users.

Non-functional requirements and verification

ID	Category	Measurable interpretation	Verification and current limit
NFR-01	Performance	Core page responses should finish within 3 seconds on the local demo under ordinary single-user use	See performance evidence. Not a production concurrency guarantee
NFR-02	Usability	Desktop and phone layouts remain usable; fields have labels; status includes text; keyboard focus is visible; reduced-motion preferences are respected	Browser review at desktop and 390 px. A formal WCAG audit and stakeholder usability study remain pending
NFR-03	Reliability	Rejected requests do not leave partial bookings/slots; errors explain how to retry	Transaction rollback, validation and collision tests
NFR-04	Security	Hash passwords; restrict roles and record ownership; retain CSRF protection	Security tests, policies and middleware
NFR-05	Maintainability	Separate HTTP, domain operations, data models and templates; reproducible tests and format checks	Laravel structure, BookingService, PHPUnit, Pint and lockfiles

ID	Category	Measurable interpretation	Verification and current limit
NFR-06	Availability	The app is available while its demonstration host/process is running; restart preserves records	File-backed SQLite and launcher. No uptime SLA
NFR-07	Portability	A documented container configuration reproduces the runtime and storage structure	Dockerfile/Compose supplied; Docker execution remains unverified on this host

Business rules

• Roles are customer, provider and admin. Registration never accepts admin.

• Browse is public; booking creation requires an active customer.

• A listing is bookable only when it and its provider are active.

• A provider has at most one reserved appointment for a given start time across all their listings.

• Five non-overlapping two-hour start slots operate daily. No holidays or custom weekly availability are implemented.

• Quote and service title are copied into bookings at request time.

• Provider transitions: pending -> accepted/rejected; accepted -> in_progress; in_progress -> completed.

• Customer cancellation applies only to pending/accepted bookings with start >= now + two hours.

• Completed, rejected and cancelled states cannot be reopened.

• Rejection/cancellation frees the reserved slot. Completed bookings remain history.

• Reviews require the matching customer, completed status, integer rating 1-5 and a 10-1000 character comment.

• Administrators cannot suspend another administrator or themselves.

• HTML input is displayed as escaped text.

Use cases

ID	Actor / precondition	Main flow	Alternate / result
UC-01 Register	Visitor	Enter details, select customer/provider, submit, create hashed account, sign in	Invalid fields or duplicate email keep the form and show errors
UC-02 Aut henticate	Active account holder	Submit credentials, regenerate session, open role dashboard	Incorrect or suspended account returns a generic credential error; repeated attempts are limited
UC-03 Manage service	Provider signed in	Create category/title/description/price/city listing; edit, pause or remove owned listing	Wrong owner is forbidden; soft deletion preserves bookings

ID	Actor / precondition	Main flow	Alternate / result
UC-04 Discover	Any visitor	Enter text/city, choose category and sort, inspect results	Empty state offers clear filters
UC-05 View details	Active listing	Open service; read provider, price note and completed-booking reviews	Hidden/removed service returns not found
UC-06 Request booking	Customer and active listing	Choose date and slot, enter address/phone, validate, persist booking + reservation + event atomically	Collision or invalid data leaves no partial request
UC-07 Process booking	Assigned provider	Review details; accept/reject; start; complete	Wrong provider is forbidden; stale/invalid transitions show errors
UC-08 Review history	Signed-in customer/provider	List owned requests; filter status; open detail/timeline	Another ordinary user cannot inspect contact details
UC-09 Manage accounts	Administrator	Search, inspect account role/status, suspend or reactivate	Admin accounts remain protected
UC-10 Maintain profile	Signed-in user	Update personal details and bio; change password with current password	Role injection is ignored; duplicate email/incorrect current password rejected
UC-11 Submit review	Customer, completed owned booking	Enter rating/comment; save once; show on service	Unfinished/wrong-owner/duplicate review rejected
UC-12 Cancel	Customer, owned eligible booking	Confirm cancellation; transactional state change, slot deletion and event	Inside two hours or terminal state leaves records unchanged

Design linkage

Identity and roles map to the architecture’s web middleware and policy layer. FR-05-FR-09 map to Category, ServiceListing and User in the domain/ER diagrams. FR-10-FR-13 and FR-18-FR-20 map to the booking sequence, lifecycle activity, Booking, BookingSlot, BookingEvent and Review. UC-12 is shown explicitly in the maintenance flow. Test case IDs and expected/actual results are in TESTING.md and the generated test register.

Validation record

Requirements derived from the provided proposal and refined during implementation. Group stakeholder review: pending. Lecturer acceptance of maintenance CR-01: pending. Record actual reviewers, dates and changes in CONTRIBUTIONS.md.

Agile plan, backlog and acceptance criteria

Group 3 · Five-person team · Planning artifact for team adoption.

The proposal specifies Agile with five increments. The schedule below is a recommended plan, not a fabricated sprint history. Assign actual names, dates and evidence after team review.

Product backlog

ID	Scope	Priority	Estimate	Dependencies
PB-01	Accounts, sessions, profiles and roles	Must	5 points	None
PB-02	Provider profiles and service management	Must	5	PB-01
PB-03	Browse, search, category/city filtering and details	Must	5	PB-02
PB-04	Requests, conflict prevention and booking lifecycle	Must	8	PB-01, PB-02
PB-05	Administrative accounts and oversight	Should	3	PB-01
PB-06	Validation, error handling and security integration	Must	3	Cross-cutting
PB-07	Completed-booking reviews	Should	3	PB-04
PB-08	Customer cancellation, CR-01	Should	3	Tested PB-04 baseline

User stories

ID	As a… I want… so that…	Acceptance criteria
US-01	customer/provider, to register and sign in securely, so I reach my workspace	Valid customer/provider details create an account; passwords are hashed; invalid/admin registration is rejected; session regenerates on login
US-02	provider, to introduce myself and publish services, so customers can understand my work	Profile bio is shown; category/title/description/price/city are required; listing belongs to the signed-in provider
US-03	customer, to browse/filter services, so I find relevant help	Text, category and city combine; inactive services/providers are hidden; sort choices are restricted

ID	As a… I want… so that…	Acceptance criteria
US-04	customer, to request a date/time, so I can plan a service	Future allowed slot plus valid address/phone creates pending request; quote is saved; conflicting slot produces an error and no partial records
US-05	provider, to respond to requests and record progress, so customers know what happens next	Only assigned provider may act; pending accepts/rejects; accepted starts; in-progress completes; every change has an actor/time
US-06	administrator, to list and search accounts, so I can operate the platform	Admin dashboard shows account search, role/status and activity; ordinary users receive forbidden
US-07	user, to receive clear form errors, so I can fix my input	Errors appear near the main content; safe previous input is retained; invalid requests do not mutate records
US-08	customer, to inspect details before booking, so I can make an informed choice	Service scope, provider, city, starting price and pricing note appear; reviews originate from completed bookings
US-09	provider, to edit/pause/remove my listings, so information remains current	Ownership is enforced; pausing hides catalog availability; removal keeps historical booking details
US-10	account holder, to have role-specific access, so private data and operations stay protected	Registration cannot grant admin; profile submission cannot change role; policies block cross-user booking/listing access
US-11	customer/provider, to see booking history and status, so I can follow the work	Own booking list is paginated/filterable; a detail timeline displays all transitions; private contact details require ownership/admin
US-12	administrator, to suspend/reactivate an ordinary user, so misuse can be managed	Suspension ends the next authenticated request and hides provider services; admin accounts are protected; reactivation restores access
US-13	customer, to review completed work, so others can choose with useful feedback	Matching customer may submit one 1-5 rating with a valid comment; unfinished, duplicate and wrong-owner reviews fail
US-14	customer, to cancel an eligible appointment, so changed plans free the provider’s time	Explicit confirmation required; owner only; pending/accepted and >=2 hours ahead; slot released and history kept

Recommended sprint plan

Sprint	Goal and working increment	Planned owners	Review evidence
1	Identity, requirements, roles and profile	Members 1+2; member 3 reviews	Auth demo, initial requirements and auth tests

Sprint	Goal and working increment	Planned owners	Review evidence
2	Provider listings and domain/database design	Members 2+3; member 4 reviews	Listing CRUD, migration, ER and ownership tests
3	Discovery and responsive UI	Members 3+4; member 5 reviews	Search demo, interface prototype, keyboard/mobile checks
4	Booking requests and state workflow	Members 4+5; member 1 reviews	End-to-end demo, sequences and collision tests
5	Administration, review, deployment, security	All; rotating reviewer	Test report, deploy guide, risk register and acceptance record
Maintenance	Lecturer-provided/approved change	Assign after approval	Baseline diff, impact analysis, regression evidence

Suggested cadence: one week per sprint, 15-minute daily coordination, backlog refinement twice per sprint, review and retrospective at the end. Dates must follow the actual course calendar.

Definition of ready

A story has an actor, business outcome, acceptance criteria, priority and dependencies; privacy and validation expectations are understood; the team can explain the design.

Definition of done

Code is reviewed; relevant automated tests pass; important browser paths work; no secrets are committed; requirements/design/user guides are updated; the increment can be demonstrated; actual contributors and reviewers are recorded.

Team working agreement

Use a branch per change, descriptive commits and reviewed pull requests once your GitHub repository is created. Commit real work under the member’s own identity. Rotate design, implementation, testing and demonstration responsibilities. Do not report these suggested activities as completed without evidence.

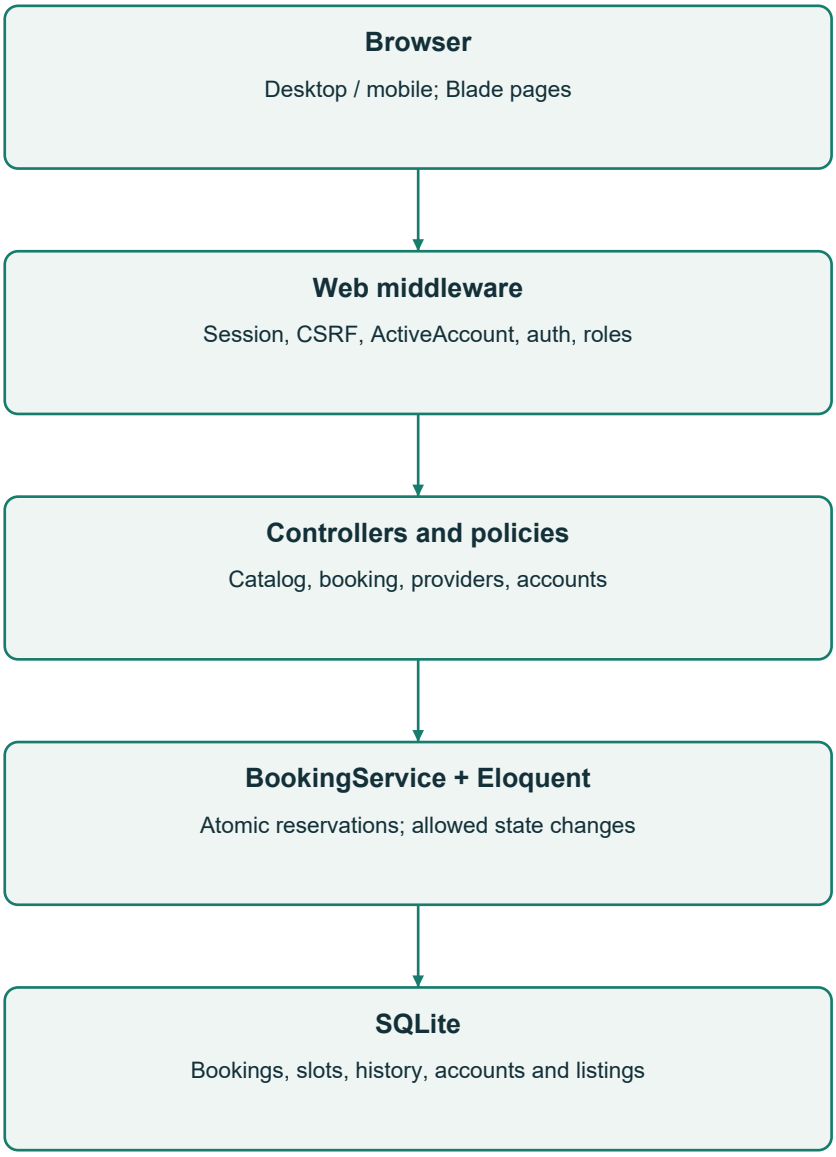
Actual build record

The initial local implementation and baseline tests were produced with AI assistance. Local commits identify that origin. The separate CR-01 increment follows the tested baseline. No team meetings, sprint burndown, stakeholder sign-off or GitHub collaboration history are invented.

System design diagrams

These diagrams summarize the implementation. Complete editable Mermaid sources, entity fields, cardinalities, sequence alternatives and wireframes are in docs/DESIGN.md. Framework session/cache/job tables are omitted from the domain view.

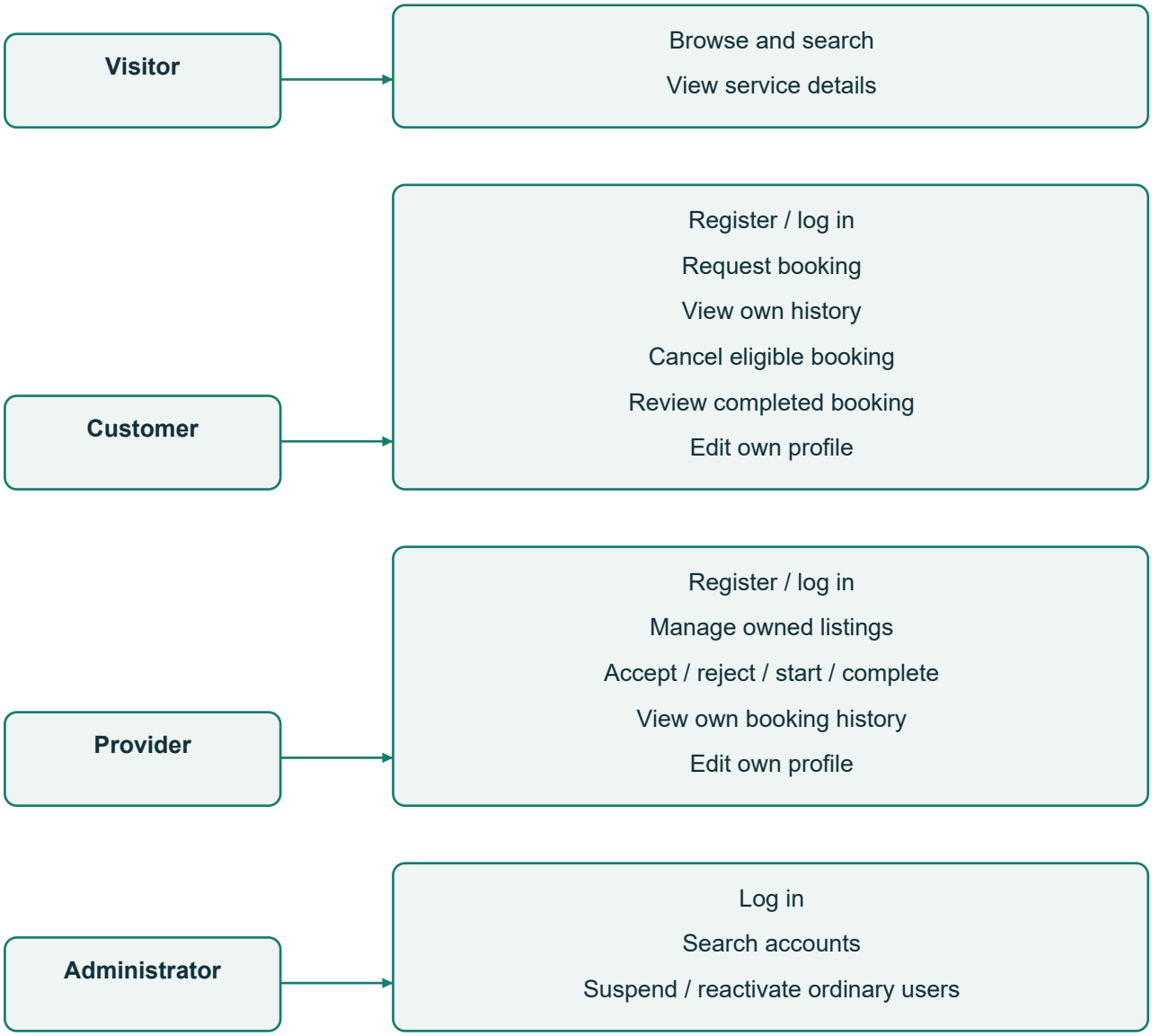
Architecture



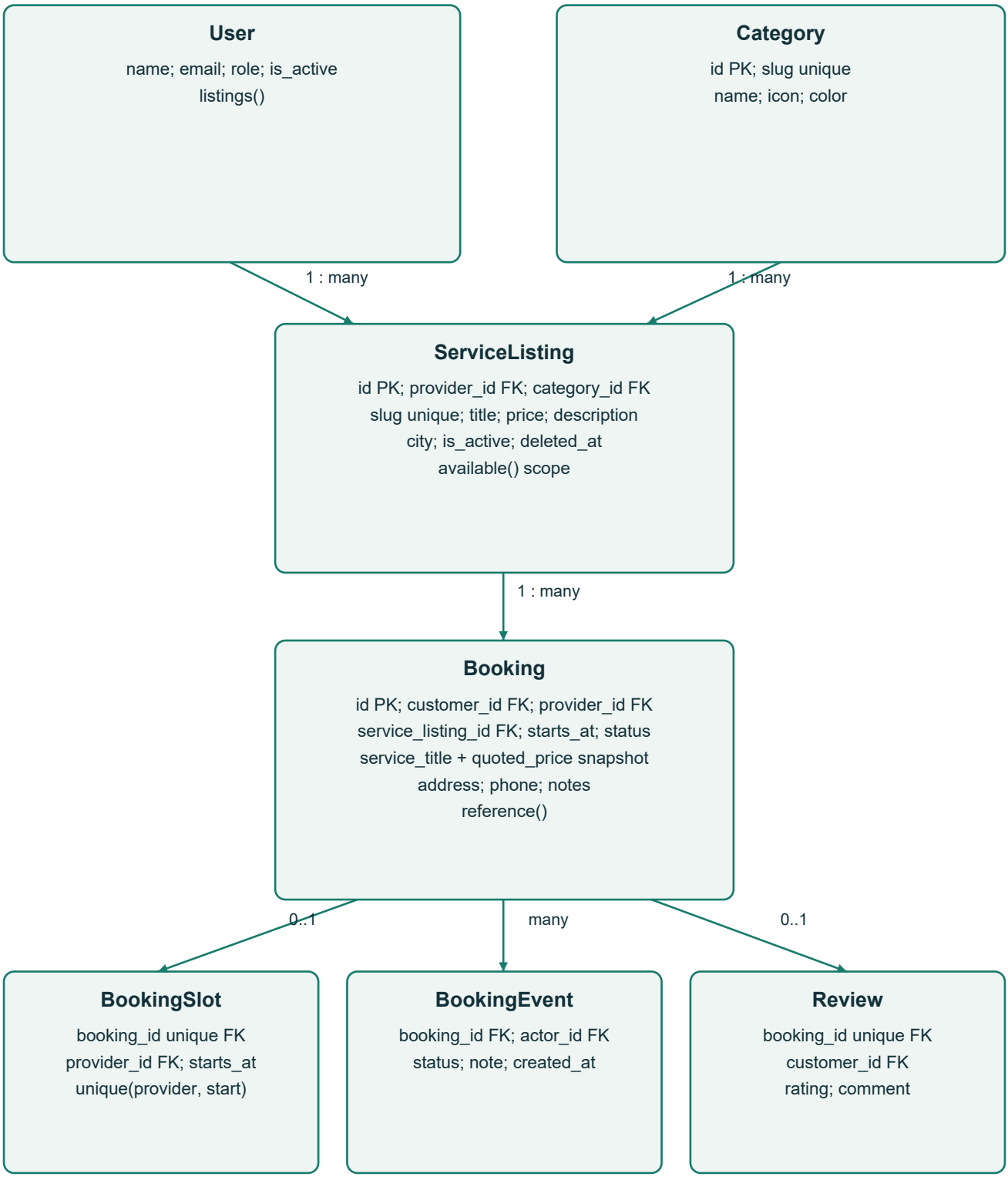
Assets: Vite + Tailwind + Alpine + GSAP + local fonts

Slot lookup returns unavailable times only. Pages render server-side.

Use cases by role

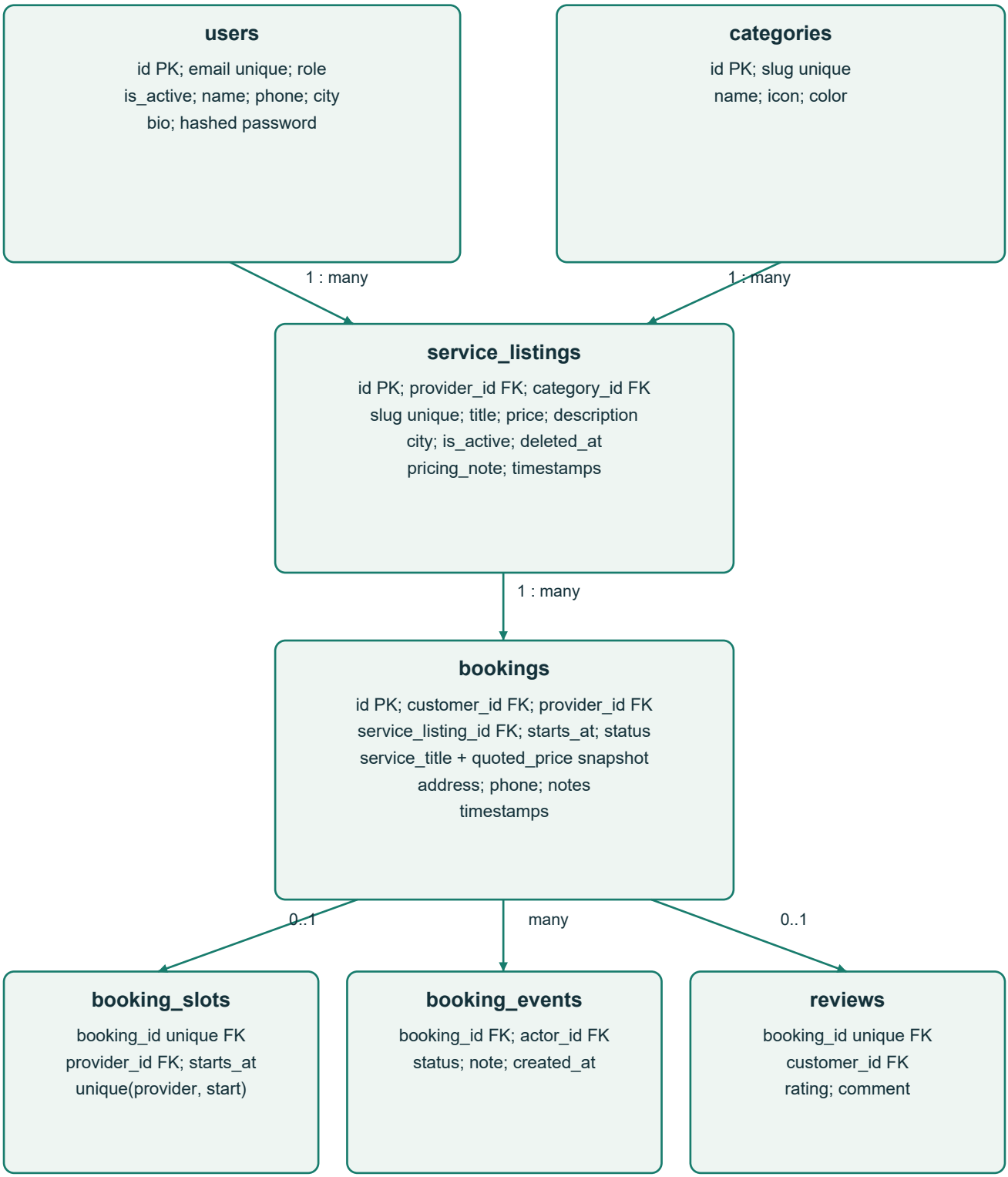


Domain and class relationships



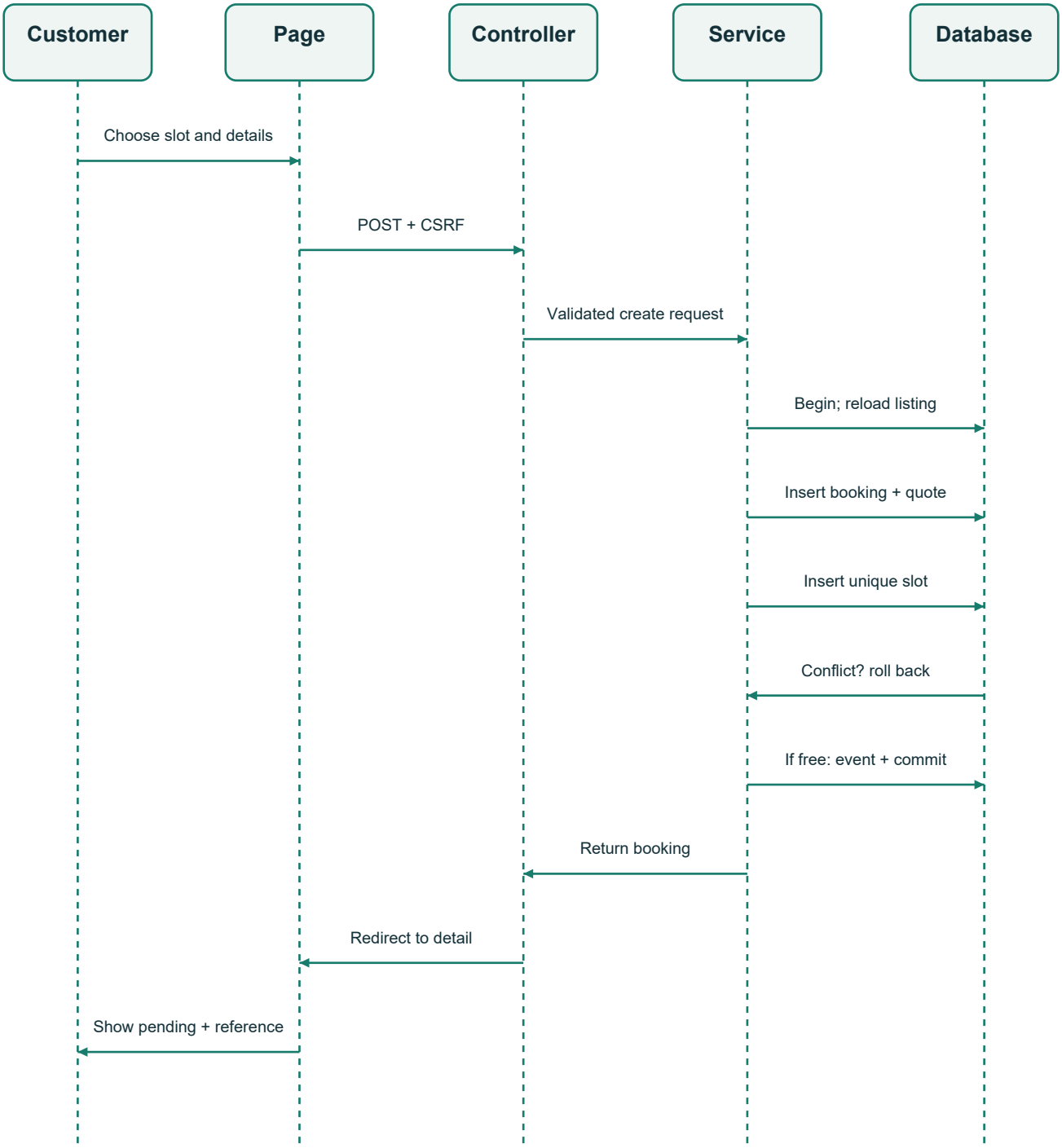
Users also relate to bookings as customer/provider, events as actor, and reviews as author. BookingService.create(), transition() and cancel() coordinate transactions. BookingSlot is accessed through the query builder. See editable sources for the complete graph.

Database / ER relationships



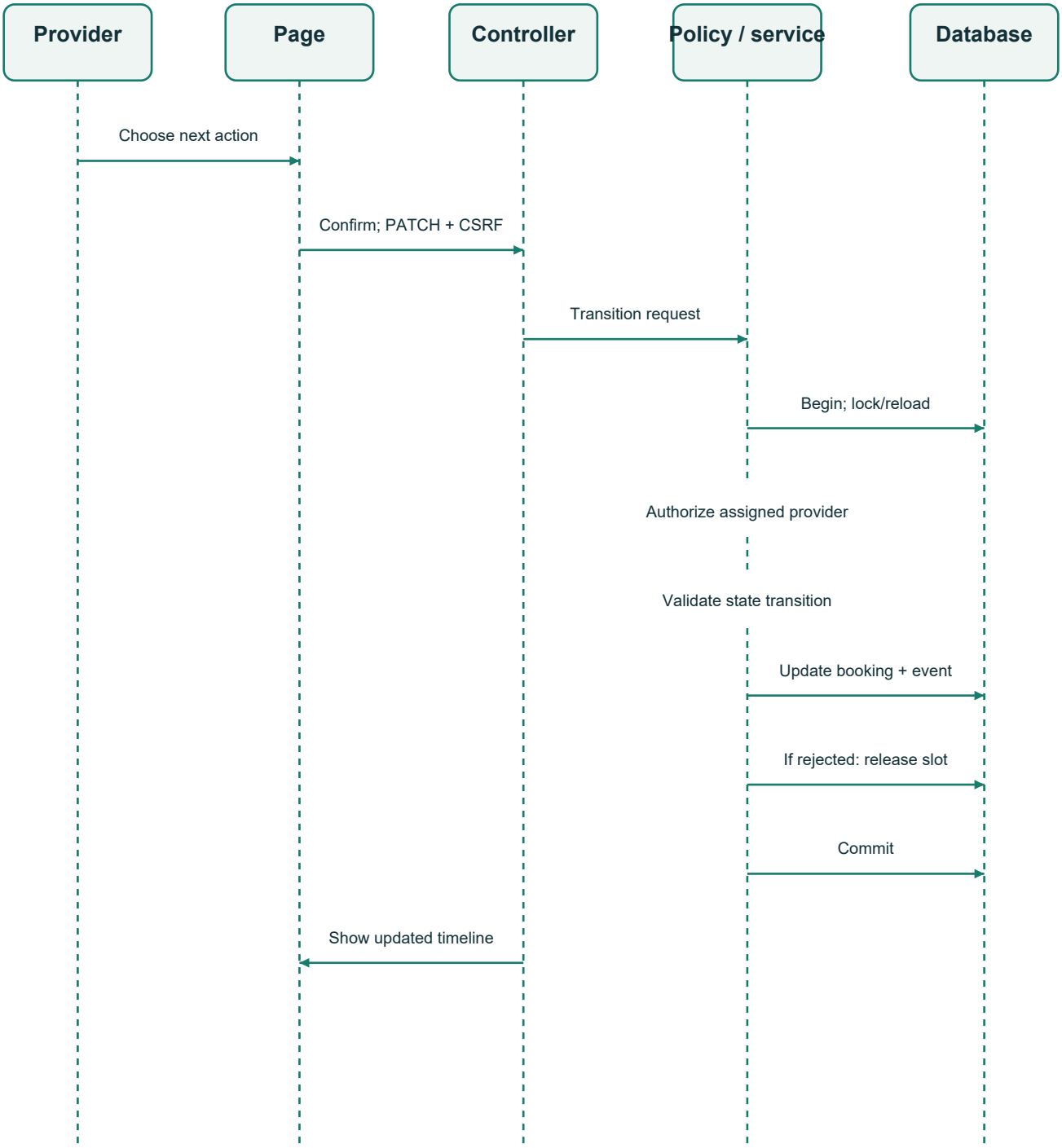
Users also relate to bookings as customer/provider, events as actor, and reviews as author. BookingService.create(), transition() and cancel() coordinate transactions. BookingSlot is accessed through the query builder. See editable sources for the complete graph.

Sequence: request a booking

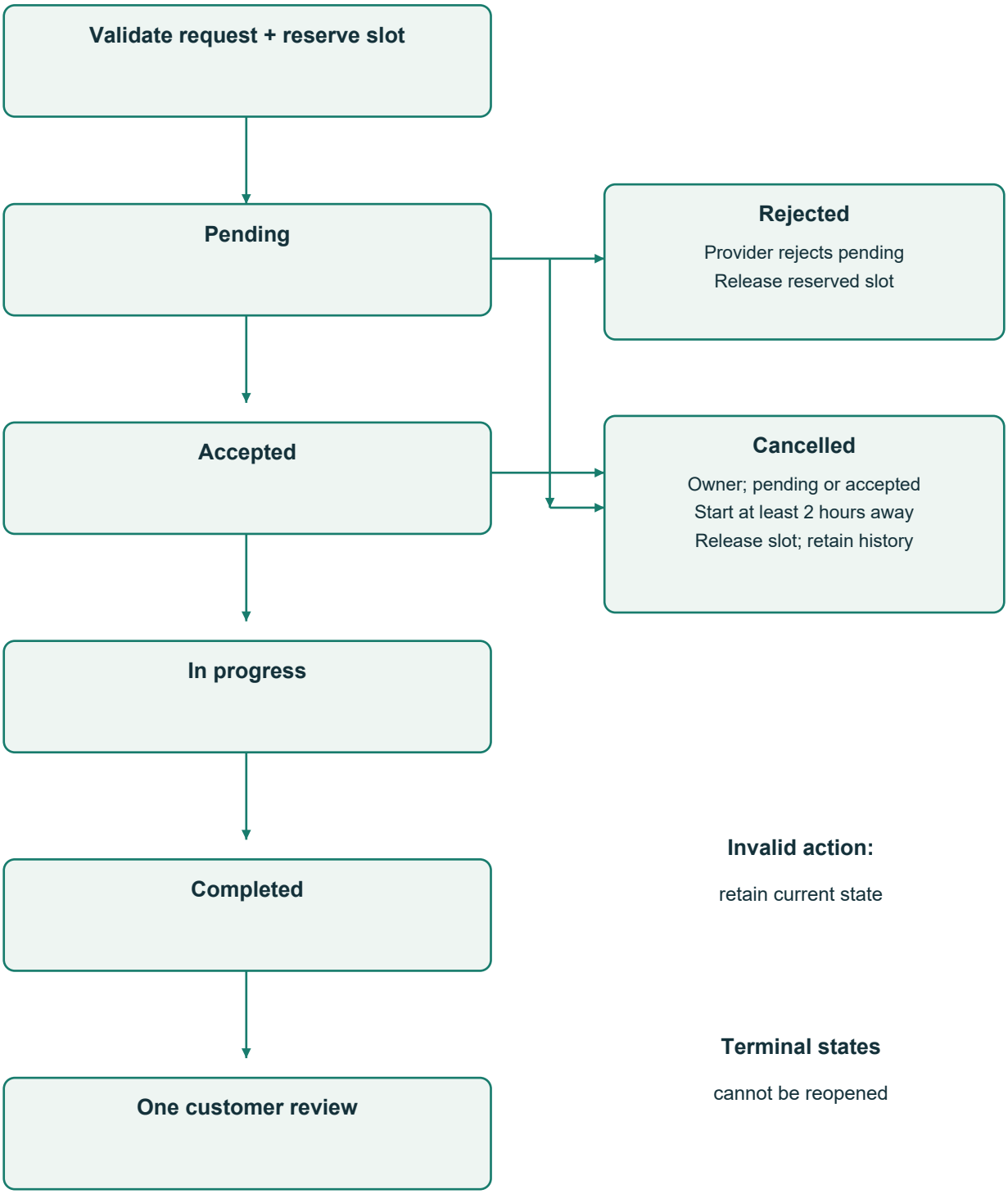


Alternative: a slot uniqueness failure rolls back the booking and returns a validation error; no partial booking remains.

Sequence: provider response



Activity: booking lifecycle



Validation failure returns to the form. Rejected, cancelled and completed records stay in history. Only completed owned bookings can receive one review.

Security assessment

Scope: HomeLink local academic demo and supplied deployment configuration. This is a practical assessment, not a penetration-test certification.

Risk register

Likelihood and impact are Low / Medium / High.

Threat	Likelihood	Impact	Controls implemented	Residual issue
Credential guessing	High	High	Hashed passwords; 10+ character mixed-case/numeric registration; login throttling	No MFA, password-reset delivery or breached-password check
Privilege escalation	Medium	High	Customer/provider registration allow-list; profile ignores role; role middleware	Admin provisioning must remain an operator action
Insecure direct object reference	High	High	BookingPolicy and ServiceListingPolicy; ownership-scoped history	Authorized admins can see platform booking details
Request forgery	Medium	High	Laravel web request-forgery middleware; CSRF token on every mutation	Deployment must use secure same-site cookies and HTTPS
Stored/reflected XSS	Medium	High	Escaped Blade interpolation; no untrusted raw HTML	No full content-security-policy rollout; Alpine requires a considered CSP strategy
SQL injection / malicious sorting	Medium	High	Bound ORM queries and explicit sort allow-list	Wildcard search is basic; large catalogs need indexed search
Double booking / race	Medium	High	Unique provider/time constraint; atomic booking/slot/event creation; transaction retries	Real multi-process and alternate-database load tests remain pending
Session misuse after suspension	Medium	High	ActiveAccount logs out suspended accounts on next request; hidden provider services	Already-delivered page data cannot be recalled
Private contact-data exposure	Medium	High	Authentication + owner/provider/admin policy; slot endpoint returns only times	Local disk and backups need OS access controls

Threat	Likelihood	Impact	Controls implemented	Residual issue
Secret/demo-account exposure	Medium	High	.env and runtimes ignored; local-only listener; production seeding guard	Demo credentials are public within this package and must never be reused in production
Accidental history loss	Medium	Medium	Soft-deleted listings; preserved user rows; quote snapshots	Backup and restore procedures need operational rehearsal
Dependency vulnerabilities	Medium	High	Lockfiles; Composer/npm audit; generated CI audit step	Audits describe current advisories, not future safety

Implemented security requirements

SR-01: public registration cannot choose administrator. SR-02: all mutations require request-forgery protection. SR-03: providers can change only their own listings and bookings. SR-04: customers can view/cancel/review only their own eligible bookings. SR-05: invalid input cannot change privileged attributes or create partial records. SR-06: active sessions of suspended accounts are rejected on the next request. SR-07: passwords are hashed and never included in user serialization. SR-08: public service views escape untrusted text and exclude private contact details.

Evidence

AuthControllerTest covers weak credentials, administrator-registration rejection, profile role injection, account suspension, throttling and CSRF protection with the testing bypass disabled. CatalogControllerTest covers escaping and sort injection. BookingControllerTest covers price tampering, duplicate slots, wrong-provider updates and unauthorized detail access. ProviderControllerTest covers listing ownership. CancellationTest covers cancellation ownership and timing.

Headers include X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN and a strict-origin referrer policy. The permissions policy disables camera, microphone and geolocation because the app does not use them.

Release controls

Use APP_ENV=production, APP_DEBUG=false, HTTPS and SESSION_SECURE_COOKIE=true for public hosting. Use a fresh application key and real credentials; never publish the demo SQLite database. Keep the web root at public/. Protect backups and logs; restrict database and storage permissions. Choose a retention policy for addresses, phone numbers and history with the actual operator before handling real people’s data.

Known limitations

No provider identity verification, emergency-service guarantee, payment processing, fraud-detection service, formal accessibility certification or legal/compliance certification is implied. TLS, backup monitoring and public hosting are deployment responsibilities that have not been activated in this local delivery.

CR-01 - Customer cancellation

Status: implemented proposed change; lecturer approval pending. Classification: perfective maintenance. Baseline: local Git commit c0fd7c9, 79 passing tests.

Request

Allow a customer whose plans have changed to cancel their own pending or accepted booking at least two hours before the scheduled start. Free the provider's reserved time and retain the historical record.

This example was selected during implementation. The assignment requires a lecturer-provided or approved request, so obtain actual approval before claiming this satisfies the assessed maintenance activity.

Before and after

Before: a customer could track requests, but only providers could change their status. After: the customer sees cancellation controls for eligible bookings, confirms the action, and receives a cancelled status and timeline event. The reserved slot becomes bookable again.

Acceptance criteria

1. Only the booking's customer may cancel. 2. Eligible states: pending and accepted. 3. Start time must be >= current time + two hours. The exact boundary is allowed. 4. The action is atomic: state, slot and timeline change together. 5. Ineligible/unauthorized requests leave records unchanged. 6. Repeating cancellation does not add another event. 7. Provider workflow cannot reopen a cancelled booking. 8. Booking list, detail, user guide and traceability include cancellation.

Impact analysis

Area	Impact
Requirements	Adds FR-19 / US-14 / UC-12; refines booking-management scope
Design	Adds cancellation branch in D-07; uses existing status enum case and transaction boundary
Database	No schema migration required; existing cancelled state and reservation/event tables suffice
Code	BookingService::cancel, BookingPolicy::cancel, BookingController::cancel, named POST route
UI	Conditional action, confirmation, closed-window guidance and status badge
Tests	Nine regression cases in CancellationTest
Documentation	Requirements, activity diagram, user guide, demo and this record

Area	Impact
Operations	No external service, data migration or new dependency

Estimate

Planning estimate: approximately 0.5-1 team day including review and regression work. This is not a claim of measured student hours.

Implementation and rollback

The operation starts a transaction, reloads/locks the booking, checks ownership and current eligibility, changes status, removes the slot and appends an actor-stamped event. State checks occur inside the transaction so a stale page does not authorize an obsolete operation.

For code rollback, revert the CR-01 commit through a reviewed Git revert. Existing cancelled bookings must remain readable; do not rewrite their history or silently rebook slots. No database rollback is required.

Test result

Nine cancellation tests pass. Full regression result: 88 tests, 241 assertions. Cases cover normal cancellation, the exact cutoff, late requests, another customer, four ineligible states and rebooking the freed slot. See evidence/final-junit.xml.

Approval record

Lecturer name/date/sign-off: pending. Group reviewer and evidence: pending. If the lecturer supplies a different request, create CR-02, preserve this record, and perform a fresh impact analysis and regression run.

Test plan and execution report

Scope and strategy

Verify the prioritized functional requirements, security boundaries, booking transaction invariants and critical UI workflows.

• **Unit:** the permitted BookingStatus transitions.

• **Integration:** HTTP requests through Laravel middleware/controllers/policies/services against real in-memory SQLite.

• **System:** live browser navigation, login, request creation and provider processing in the local app.

• **Acceptance:** executable scenarios derived from user-story criteria; independent team/stakeholder sign-off remains pending.

Tests must not overwrite the demo database. Tests/TestCase uses LazilyRefreshDatabase; phpunit.xml selects :memory:. Password hashing cost is reduced only in testing. No external service is mocked as a completed integration because none is required.

Entry and exit criteria

Entry: migrations succeed, dependencies install, frontend builds and seed data is available. Exit: all implemented critical-path tests pass, negative cases preserve records, view compilation succeeds and important screens are inspected. Public-host/Docker/CI approval is a separate unexecuted stage.

Recorded results

Baseline: 79 tests, 213 assertions, all passed. After CR-01: 88 tests, 241 assertions, all passed. Machine-readable evidence: evidence/baseline-junit.xml and evidence/final-junit.xml. Detailed generated case register: TEST-REGISTER.md, including test identifiers, actual outcome and requirement mapping.

The baseline run took approximately 4.3 seconds and the post-change run approximately 5.2 seconds in this environment. These measure the automated suite, not website loading time.

Representative test cases

Case	Requirement	Preconditions / input	Expected result	Actual result
TC-AUTH-01	FR-01	Valid customer registration	Account saved, hashed password, signed in	Passed
TC-AUTH-02	FR-01, FR-17	Register with role=admin	Error, no account	Passed
TC-AUTH-03	FR-02	Wrong password / suspended user	Generic credential error, guest remains	Passed
TC-PROFILE-01	FR-04, FR-17	Valid edit plus role/is_active injection	Allowed details update; privilege unchanged	Passed

Case	Requirement	Preconditions / input	Expected result	Actual result
TC-SERVICE-01	FR-05	Provider creates valid listing	Listing belongs to provider	Passed
TC-SERVICE-02	FR-06, FR-17	Another provider edits/deletes listing	Forbidden; original preserved	Passed
TC-SEARCH-01	FR-07-09	Combined text/category/city query	Only matching active listings	Passed
TC-BOOK-01	FR-10	Future valid request with forged price	Pending booking; server quote; slot and event saved	Passed
TC-BOOK-02	FR-10, FR-16	Same provider/time through another listing	Validation error and transaction rollback	Passed
TC-BOOK-03	FR-10	Past/too-distant date or bad contact fields	Error; no booking/slot	Passed
TC-STATUS-01	FR-13, FR-20	Assigned provider accepts then processes	Valid state and timeline changes	Passed
TC-STATUS-02	FR-13	Attempt pending -> completed	Error; pending retained	Passed
TC-PRIVACY-01	FR-17	Other customer opens booking	Forbidden; contact details absent	Passed
TC-REVIEW-01	FR-18	Completed booking, correct customer	One valid review persists	Passed
TC-REVIEW-02	FR-18	Duplicate / unfinished / wrong owner	Error or forbidden; no extra review	Passed
TC-ADMIN-01	FR-14-15	Admin suspends ordinary account	Suspended state persists	Passed
TC-ADMIN-02	FR-15, FR-17	Suspend admin / ordinary user opens admin	Forbidden	Passed
TC-SEC-01	NFR-04	Missing CSRF token with bypass disabled	419; no registration	Passed
TC-SEC-02	NFR-04	Script text and malicious sort	Escaped output / validation error	Passed
TC-CR-01	FR-19	Owned pending booking >=2h ahead	Cancelled, slot released, event added	Passed
TC-CR-02	FR-19	Exact two-hour boundary	Cancellation allowed	Passed
TC-CR-03	FR-19	Late, foreign-owner or terminal booking	No unauthorized mutation	Passed
TC-CR-04	FR-19	Book the newly released slot	New reservation succeeds	Passed

Defect log

ID	Observed issue	Correction	Verification
DEF-01	Default application URL already included :8000; naive replacement duplicated the port	Replace the complete APP_URL line	Migrations, routes and live server succeeded
DEF-02	Fresh user objects did not contain database role/active defaults until reload	Explicit model attribute defaults	Previously failing auth/provider cases pass
DEF-03	GSAP attempted hero animations on routes without a hero, producing console warnings	Check the target exists before animating	Browser logs after rebuilding
DEF-04	Native confirmation dialog interrupted automated browser interaction	Accessible in-page HTML dialog with cancel/confirm controls	Provider start/completion completed through the live UI
DEF-05	Importing the entire icon collection inflated JavaScript	Import only used icons and Latin font subsets	Build reduced JS from about 510 KB to about 184 KB
DEF-06	Old slot responses could overwrite results after rapid date changes	Apply availability only for the current requested date	Source review; server still enforces conflict protection

Browser record

The local browser verified public catalog, customer sign-in, customer booking HL-00030 with fictional data, provider sign-in, acceptance, start and completion, updated timelines, and mobile booking detail at 390 x 844. Desktop layout was reviewed at the browser’s normal viewport. The mobile drawer opens and the booking detail has no horizontal page overflow. Complete additional final observations are recorded in evidence/QA-SUMMARY.md.

Reproduction

Run php artisan test --compact from homelink. For one area: php artisan test --compact tests/Feature/BookingControllerTest.php. Regenerate JUnit with --log-junit docs/evidence/final-junit.xml. Do not edit expected outcomes merely to make a failing test pass.

Limits

No real customer/provider interviews, external acceptance signatures, production concurrency benchmark, cross-database test matrix, full WCAG audit, Docker execution or remote CI execution is claimed.

Final verification and handover

Verified on 16 September 2026. The complete execution record and raw evidence are in docs/evidence/QA-SUMMARY.md and the adjacent JSON/XML files.

Automated checks

88 tests and 241 assertions passed with no failures, errors or skips. Laravel Pint, the Vite production build and Blade view compilation passed. Composer and npm reported no known dependency advisories in this point-in-time check.

Live browser checks

The customer requested booking HL-00030. The provider accepted, started and completed it; the customer then submitted a fictional demonstration review. Eligible accepted booking HL-00029 was cancelled with a confirmation dialog, and its history updated. The administrator workspace and account table were inspected.

The catalog and booking detail were inspected at desktop size and at 390 x 844. The mobile drawer opened, and booking detail had no page-level horizontal overflow. Hidden mobile navigation uses inert and aria-hidden bindings. The latest review tab had no captured console errors or warnings.

Local performance observation

Twenty sequential local HTML requests had a median of 255.22 ms and a maximum of 401.88 ms. This excludes browser assets, rendering and concurrent users; it does not establish a production three-second page-load guarantee.

Run and present

Open Desktop/SE_project and launch START HOMELINK.cmd. Browse http://127.0.0.1:8000. The user guide contains customer, provider and administrator demo credentials. Use the editable presentation and docs/DEMO.md for the demonstration.

Complete the human evidence

Add actual member names, student IDs and contribution evidence. Validate requirements with stakeholders and obtain lecturer approval for the maintenance change. Docker execution, remote CI, public hosting, MySQL integration and a full accessibility audit remain unverified.